دليل مبسّط وموسّع للمناقشة

منصة ذكاء أعمال معتمدة على الوكلاء الذكيّة

An AI-Agent-Based Business Intelligence Platform for Analyzing Databases and Documents

شرح الفكرة والمعمارية والوحدات ومسار السؤال
مع توضيح المصطلحات التقنية وأسئلة المناقشة

كيف تستخدم هذا الدليل؟

هذا الدليل يعيد شرح المحتوى المرفق بلغة عربية أوضح، ويشرح لماذا توجد المكونات وكيف تتعاون، بدل الاكتفاء بسرد أسماء الملفات. أبقينا أسماء الوحدات والملفات الإنجليزية كما هي لتسهيل مطابقتها مع الشيفرة.

> **تنبيه مهم:** الشرح مبني على وصف وبنية المشروع الواردة في النص المرفق. لا يُعدّ تدقيقًا مباشرًا للشيفرة؛ لذلك عند عرض تفاصيل تنفيذية دقيقة، طابقها مع الملفات الفعلية. ووفق الوصف، قاعدة الإنتاج المقصودة هي Supabase / PostgreSQL، أما MongoDB فاستُخدم للاختبار أو كمحوّل مؤقت.

الفهرس

1. الفكرة العامة وما الذي تحله المنصة
2. الصورة المعمارية ومفاهيم الطبقات
3. ملفات تشغيل المشروع
4. الوحدات الست الأساسية
5. الفرق بين Agent وLLM وQuery Engine وقاعدة البيانات
6. رحلة السؤال خطوة بخطوة
7. قاعدة البيانات والوثائق وRAG
8. الأمان والاختبارات والتشغيل
9. أسئلة مناقشة وإجابات نموذجية
10. قاموس المصطلحات والخلاصة

1. الفكرة العامة: ماذا تبني المنصة؟

المشروع ليس مجرد نافذة محادثة ترسل سؤالًا إلى قاعدة البيانات. إنه نظام **ذكاء أعمال** يستقبل سؤالًا باللغة الطبيعية، يحدد ما المطلوب، ويختار أداة مناسبة لجلب البيانات أو البحث في الوثائق، ثم يحلل النتائج ويعيدها في صورة تقرير أو إجابة مفهومة.

```
مستخدم يسأل سؤالًا
        ↓
وكيل ذكي يحدد المطلوب ويخطط
        ↓
أداة قاعدة بيانات و/أو أداة استرجاع وثائق
        ↓
جمع النتائج وتحليلها
        ↓
تقرير واضح للمستخدم
```

مثال: «حلّل مبيعات يناير وقارنها بديسمبر». هذا الطلب قد يتطلب أكثر من خطوة: استخراج بيانات الشهرين، حساب الفروقات، تفسير النتائج، ثم كتابة ملخص. لذلك يحتاج النظام إلى التخطيط والتنفيذ والتحليل، لا إلى توليد جملة فحسب.

مصطلح ذكاء الأعمال

Business Intelligence (BI) هو تحويل البيانات المتاحة إلى معلومات تساعد على الفهم واتخاذ القرار: أعداد، مقارنات، اتجاهات، مؤشرات، واستنتاجات.

2. الصورة المعمارية والمبدأ التصميمي

بحسب بنية المشروع المذكورة، تنقسم الوظائف إلى وحدات مستقلة؛ لكل وحدة مسؤولية محددة، وتتواصل الوحدات عبر خدمات وعقود واضحة.

```
Frontend → FastAPI → Multi-Agent
                     ├─ DatabaseTool → Query Engine → Chat Database → Supabase/
PostgreSQL
                     └─ RAGTool      → RAG Service   → Vector DB → Documents

LLM Module: واجهة موحّدة للنماذج اللغوية
Memory: سياق المحادثة والمعلومات المفيدة
النتيجة: تحليل / رؤى / تقرير
```

فصل المسؤوليات

Separation of Concerns يعني ألا تُحشر كل المسؤوليات في ملف أو مكوّن واحد. الوكيل يقرر، ومحرك الاستعلام ينسق الاستعلام، ووحدة قاعدة البيانات تتعامل مع SQL والمصدر، ووحدة المخطط تشرح بنية الجداول، وطبقة النموذج تتصل بمزوّد LLM. هذا يسهل فهم النظام واختباره وتغييره.

الطبقات المعتادة داخل الوحدة

الطبقة	معناها ببساطة	مثال دورها
Domain	مفاهيم وقواعد المجال الأساسية، من دون تفاصيل البنية التحتية	تعريف الاستعلام أو المهمة أو عقد الواجهة
Application	تنسيق حالات الاستخدام وتسلسل العمل	تنفيذ خدمة الاستعلام أو إنشاء الخطة
Infrastructure	تفاصيل الاتصال والتنفيذ الفعلي	موصل PostgreSQL أو مزود Gemini
Interfaces	طرق دخول النظام أو تقديم الخدمة	مسارات API وبيانات الطلب والاستجابة

قاعدة سهلة: المجال يعرّف ما الذي يعنيه الشيء، التطبيق ينسق ما ينبغي فعله، البنية التحتية تنفذ كيف يتم ذلك، والواجهات تتيح الوصول إليه.

3. ملفات المشروع خارج الوحدات

- **.gitignore:** يحدد الملفات التي لا ينبغي إدراجها في Git، مثل أسرار البيئة والملفات المؤقتة وملفات Python المولّدة. لا تضع مفاتيح API في مستودع الشيفرة.
- **README.md:** دليل للمطور أو المستخدم الجديد: ما المشروع، متطلباته، طريقة تشغيله، وواجهاته.
- **requirements.txt:** قائمة مكتبات Python المطلوبة، ويمكن تثبيتها عادةً باستخدام `pip install -r requirements.txt`.
- **render.yaml:** وصف إعداد نشر الخدمة على Render، بحسب إعداد المشروع.
- **structure.txt:** توثيق نصي لشجرة الملفات، وليس منطق التطبيق نفسه.
- **src/main.py:** نقطة دخول تطبيق FastAPI، تُنشأ فيها عادةً كائنات التطبيق وتُسجل مسارات API.
- **src/bootstrap.py:** يجمع المكونات ويربط اعتمادياتها. هذا يسمى Dependency Injection / Composition Root.
- **src/constants.py:** مكان مركزي للثوابت التي تشترك فيها أجزاء المشروع.
- **src/run.bat:** ملف تشغيل مساعد على Windows.

تشبيه: إذا كانت الوحدات قطع سيارة، فإن bootstrap.py يركّب القطع ويوصلها، وmain.py هو المفتاح الذي يبدأ تشغيل التطبيق ويعرض منافذه.

4. الوحدات الأساسية في المشروع

4.1 وحدة chat_database: من اللغة الطبيعية إلى نتيجة قاعدة البيانات

مسؤوليتها المساعدة على تحويل السؤال إلى استعلام قاعدة بيانات، التحقق منه، تنفيذه، ثم إعادة نتيجة منظمة. لا تساوي قاعدة البيانات نفسها؛ إنها جزء التطبيق الذي يتعامل معها.

- **application/dtos.py:** كائنات نقل البيانات DTO، مثل تمثيل السؤال والمخطط وSQL والنتيجة بين الطبقات.
- **prompt_builder.py:** يجمع سؤال المستخدم وقواعد SQL ومعلومات المخطط في تعليمات مناسبة للنموذج.
- **services.py:** ينسق توليد SQL والتحقق والتنفيذ والتنسيق، حتى لا تضطر API لإدارة كل تفصيل.
- **domain/:** كيانات وقيم وعقود وأخطاء خاصة بالمجال.
- **llm_sql_generator.py:** يطلب من النموذج توليد SQL استنادًا إلى السؤال والسياق.
- **sql_validator.py:** يفحص الاستعلام قبل التنفيذ وفق سياسة الأمان المعتمدة.
- **postgres_executor.py:** ينفذ الاستعلام على PostgreSQL بحسب الوصف.
- **result_formatter.py:** يحول الصفوف والأعمدة والبيانات الوصفية إلى نتيجة مفهومة لباقي المكونات.
- **interfaces/router.py وschemas.py:** واجهة API وأشكال الطلب والاستجابة.
- **prompts/:** أمثلة وقواعد وتعليمات تساعد على توليد SQL أكثر انضباطًا.

> **الأمان:** فحص SQL مهم لكنه لا يكفي وحده. في الإنتاج يجب أيضًا تقييد صلاحيات مستخدم قاعدة البيانات، واستخدام استعلامات للقراءة فقط حيث أمكن، وتحديد الجداول والأوامر المسموحة، والتحقق من مدخلات المستخدم، وتطبيق حدود للوقت والصفوف. لا تنفذ SQL مولّدًا بلا ضوابط.

4.2 وحدة LLM: واجهة واحدة لمزوّدات متعددة

LLM هو نموذج لغوي كبير. وحدة LLM تجعل بقية النظام تتعامل مع واجهة داخلية موحدة بدل ربط كل مكوّن مباشرةً بمزوّد معين مثل Groq أو Gemini أو OpenAI-compatible أو Ollama.

- **application/services.py:** الخدمة التي تستدعيها بقية الوحدات.
- **mapper.py:** تحويل الطلب والاستجابة بين الصيغة الداخلية وصيغة المزوّد.
- **prompt_builder.py:** بناء التعليمات الموجهة للنموذج.
- **tool_calling_service.py:** دعم نمط استدعاء الأدوات عند استخدامه.
- **providers/:** تطبيقات الاتصال الفعلية بمزوّدات النماذج.
- **base_provider.py:** الواجهة أو الأساس المشترك لمزوّد.
- **provider_registry.py:** سجل المزوّدات المتاحة.
- **factory.py:** ينشئ المزوّد المناسب وفق الإعدادات.

- **health_monitor.py:** يراقب توافر الخدمات إذا كان مفعّلًا.
- **cache.py:** قد يقلل استدعاءات مكررة أو يحسن الأداء بحسب سياسة التخزين المؤقت.

الفائدة أن تغيير مزود النموذج لا يستدعي تعديل منطق الوكيل أو قاعدة البيانات بأكملها، ما دام المزوّد الجديد يلتزم بالعقد الداخلي.

4.3 وحدة Memory: ذاكرة سياق وليست قاعدة البيانات الأساسية

تساعد الذاكرة على الاحتفاظ بسياق المحادثة أو معلومات سابقة نافعة للإجابة اللاحقة. مثلًا إذا قال المستخدم «قارنها بالشهر الماضي»، قد يحتاج النظام إلى معرفة ما الذي تشير إليه «ها» من السياق السابق.

- **MemoryService:** نقطة استخدام الذاكرة من التطبيق.
- **MemoryRepository:** واجهة تخزين واسترجاع الذاكرة.
- **conversation_store.py:** إدارة سجل المحادثة أو سياقها.
- **memory_cache.py:** تخزين مؤقت لتسريع الوصول.
- **memory_retriever.py:** اختيار الذكريات ذات الصلة عند الحاجة.
- **value_objects.py:** قيم مجال مثل نوع الذاكرة أو أهميتها، بحسب النص المرفق.

> فرّق بين **ذاكرة المحادثة** التي تحفظ سياقًا مفيدًا للوكيل، وبين **قاعدة بيانات الأعمال** التي تحتوي السجلات الرسمية، وبين **Vector DB** التي قد تستخدم للبحث الدلالي في الوثائق.

4.4 وحدة Multi-Agent: التخطيط والتنسيق والعمل المتخصص

تجمع هذه الوحدة الوكلاء والخدمات التي تحوّل الطلب العام إلى خطوات وأدوار متخصصة.

- **data_agent.py:** يعمل مع البيانات أو أدواتها.
- **analyzer_agent.py:** يفسر النتائج ويستخرج المقارنات والأنماط والرؤى.
- **reporter_agent.py:** يصوغ التحليل في تقرير مفهوم.
- **agent_service.py:** المنسق الأعلى: يحمل السياق، يطلب خطة، ينفذها، ثم يحفظ المعلومات ذات الصلة إذا كان ذلك ضمن التصميم.
- **planner.py:** يحول طلبًا عالي المستوى إلى خطة قابلة للتنفيذ.
- **executor.py:** ينفذ مهام الخطة بالترتيب ويمرر السياق والنتائج.
- **bi_context.py:** سياق عمل مشترك، قد يتضمن السؤال والبيانات والتحليل والأخطاء والتقرير.
- **plan.py وtask.py:** تمثيل الخطة والمهمة المنفردة.
- **database_tool.py:** جسر إلى Query Engine، وليس قاعدة البيانات ذاتها.
- **rag_tool.py:** جسر إلى خدمة استرجاع الوثائق، وليس تنفيذ RAG كاملًا داخل الوكيل.

مثال خطة:
1) استخرج مبيعات يناير وديسمبر. 2) احسب الفرق والنسبة. 3) حلل التغيرات الملحوظة. 4) اكتب تقريرًا موجزًا مع توضيح نطاق البيانات.

4.5 وحدة Query Engine: تنظيم دورة الاستعلام

تأخذ طلبًا للاستعلام، وتبني سياقه، وتصنفه، وتوجهه إلى المحول المناسب، ثم تنفذه وتعيد نتيجة موحدة.

1. **Context Builder:** يبني سياقًا مثل المقصود ومصدر البيانات وأولوية التنفيذ.
2. **Classifier:** يحدد صنف الطلب: بيانات، مخطط، أو معلومات عن قاعدة البيانات مثلًا.
3. **Router:** يختار مسار التنفيذ أو المحول الملائم.
4. **Adapter:** يربط الواجهة العامة بمصدر محدد.
5. **Execution Manager:** يتابع حالات التنفيذ مثل قيد الانتظار، جارٍ، مكتمل، أو فاشل.
6. **Result Mapper:** يوحد شكل النتائج القادمة من مصادر مختلفة.
7. **Logger:** يسجل معلومات التشغيل المفيدة للتشخيص والتدقيق، مع مراعاة عدم تسجيل أسرار أو بيانات حساسة بلا حاجة.

وجود طبقة محولات يسمح نظريًا بإضافة مصدر أو تغييره دون أن يعرف الوكيل تفاصيل الاتصال به. ووفق الملف، قد يظهر MongoDBAdapter للاختبار، لكن قاعدة الإنتاج المقصودة هي Supabase/PostgreSQL.

4.6 وحدة Schema Engine: فهم شكل قاعدة البيانات

قبل كتابة استعلام، يحتاج النظام إلى معرفة الجداول والأعمدة وأنواعها والعلاقات بينها. هذه المعلومات تسمى Schema أو مخطط قاعدة البيانات.

- **schema_loader.py:** يجلب المخطط من المصدر.
- **metadata.py:** يتعامل مع بيانات وصفية عن قاعدة البيانات.
- **relationship_mapper.py:** يوضح صلات الجداول والمفاتيح.
- **schema_cache.py:** يحتفظ بنسخة مؤقتة لتقليل الاسترجاع المتكرر.
- **schema_formatter.py:** يصيغ المخطط بشكل يمكن تمريره إلى النموذج.
- **retriever.py:** يسترجع الجزء المطلوب من المخطط.
- **validator.py:** يتحقق من سلامة بيانات المخطط وفق قواعد النظام.
- **serializer.py:** يحول كائنات المخطط إلى صيغة قابلة للنقل أو التخزين.

مثلًا، عندما يسأل المستخدم «كم طالبًا في الذكاء الاصطناعي؟» تساعد معرفة أن جدول students يحتوي عمود department على توجيه النموذج إلى الأعمدة ذات الصلة بدل التخمين.

4.7 خدمات واختبارات وملفات مساعدة

بحسب الملف، src/services/schools_service.py خدمة خاصة ببيانات المدارس، وهي خدمة مجال/تطبيق منفصلة عن الوحدات الأساسية. مجلد test/ يحتوي اختبارات لمزودي النماذج ومراقبة صحتهم وبناء التعليمات. أما agents. و venv_backup. وpycache_backup__ فليست منطق ذكاء الأعمال الأساسي وفق الوصف؛ الأولى موارد تطوير، والأخريان نسخ احتياطية/ملفات بيئة أو تخزين مؤقت.

5. أربعة مفاهيم لا ينبغي الخلط بينها

المفهوم	السؤال الذي يجيب عنه	الدور	تشبيه
LLM	كيف أفهم أو أولد نصًا؟	يفهم اللغة ويولد استجابة أو SQL ضمن التعليمات	كاتب/محلل لغوي
Agent	ما الخطوة المناسبة لتحقيق الهدف؟	يقرر أو ينسق استخدام الأدوات والمهام	مدير يوزع العمل
Query Engine	كيف أنظم تنفيذ الاستعلام؟	يصنف ويوجه وينفذ ويعيد نتيجة منظمة	مركز توجيه للطلبات
Database	أين تحفظ البيانات الفعلية؟	تخزن السجلات وتجيب عن الاستعلامات	مستودع البيانات

> **إجابة قصيرة:** النموذج يولد ويفهم، الوكيل يخطط ويستخدم الأدوات، محرك الاستعلام يدير مسار الاستعلام، وقاعدة البيانات تحفظ البيانات.

6. رحلة السؤال من البداية إلى التقرير

السؤال التوضيحي: «كم عدد الطلاب في قسم الذكاء الاصطناعي؟» المسار التالي يشرح الفكرة المعمارية كما وردت، مع ملاحظة أن ترتيب الاستدعاءات الدقيق يجب تأكيده من الشيفرة.

1. **الواجهة:** يكتب المستخدم السؤال في الواجهة الأمامية.
2. **FastAPI:** يستقبل الطلب عبر مسار API ويتحقق من شكله الأولي.
3. **Agent Router / AgentService:** يوجه الطلب إلى خدمة الوكيل ويجهز سياق العمل.
4. **Memory:** تُسترجع معلومات سابقة ذات صلة إن وجدت، مثل سياق الجلسة.
5. **Planner:** يقرر أن المطلوب استعلام بيانات، وربما تحليل أو تقرير أيضًا.
6. **Executor:** ينفذ المهام واحدة تلو الأخرى.
7. **DatabaseTool:** يمرر طلب البيانات إلى Query Engine من دون كشف تفاصيل قاعدة البيانات للوكيل.
8. **Query Engine:** يبني السياق، يصنف الطلب ويوجهه إلى المحول المناسب.
9. **Schema Engine:** يوفر أسماء الجداول والأعمدة والعلاقات اللازمة لصياغة استعلام صحيح.
10. **Chat Database + LLM:** تُجمع تعليمات SQL والسؤال والمخطط؛ يولد النموذج SQL وفق القواعد.
11. **التحقق والتنفيذ:** يُفحص الاستعلام، ثم ينفذ على PostgreSQL/Supabase وفق الإعداد المقصود.
12. **توحيد النتيجة:** تتحول بيانات المصدر إلى QueryResult موحد.

13. **Analyzer / Reporter:** يحلل البيانات عند الحاجة ثم يصوغ إجابة أو تقريرًا.

14. **الاستجابة:** يعيد النظام النتيجة إلى المستخدم، وقد يحفظ سياقًا مفيدًا في الذاكرة.

```
السؤال → الوكيل/الخطة → الأداة → محرك الاستعلام
→ توليد + المخطط SQL → فحص SQL → PostgreSQL
→ المستخدم → تقرير → تحليل → نتيجة منظمة
```

> **تنبيه دقة:** المخطط عادةً يُسترجع قبل توليد SQL كي يُستخدم في تعليمات النموذج؛ أما الرسم النصي الأصلي فيختصر بعض الروابط. اعرض هذا كتدفق منطقي، لا كضمان لترتيب كل استدعاء داخلي، إلا بعد التحقق من تنفيذ المشروع.

7. قاعدة البيانات والوثائق وRAG

قاعدة البيانات: PostgreSQL عبر Supabase

Supabase خدمة توفر، من ضمن ما توفره، قاعدة بيانات PostgreSQL مُدارة. لذلك فإن قول «Supabase/PostgreSQL» يشير إلى خدمة Supabase التي تستضيف قاعدة PostgreSQL في المعمارية المقصودة، وليس إلى محركين متنافسين. ووفق النص، MongoDB استُخدم للتجربة أو كمحول اختباري وليس كقاعدة الإنتاج النهائية.

ما المقصود بـRAG؟

Retrieval-Augmented Generation (RAG) يعني دعم إجابة النموذج باسترجاع مقاطع ذات صلة من مستندات. بصورة عامة تُجهز الوثائق وتُجزأ، وتحول إلى تمثيلات عددية (embeddings)، ثم يُبحث عن المقاطع الأقرب لمعنى السؤال في مخزن متجهي (Vector DB). تُمرر المقاطع المسترجعة للنموذج كي يصوغ إجابة مستندة إلى محتوى الوثائق.

```
سؤال عن وثيقة → RAGTool → استرجاع المقاطع المناسبة
→ سياق موثق → LLM → ملخص / إجابة
```

الـRAG مناسب لملفات ومستندات لا تكون بالضرورة سجلات جداول. أداة قاعدة البيانات تستعلم عن بيانات منظمة، بينما أداة RAG تبحث دلاليًا في محتوى الوثائق. وقد يحتاج سؤال تحليلي إلى أحد المسارين أو كليهما.

8. لماذا هذا التصميم؟

- **سهولة التغيير:** تبديل مزود LLM أو مصدر البيانات عبر واجهة/محول بدل إعادة كتابة كل منطق الوكيل.
- **قابلية الاختبار:** اختبار المصنف أو المحول أو مخطط SQL بمعزل عن بقية النظام.
- **وضوح المسؤوليات:** تقليل التشابك وتسهيل معرفة موضع الخلل.
- **إعادة الاستخدام:** الاستفادة من الأدوات والخدمات نفسها في أكثر من مسار.

- **قابلية التوسع:** إضافة أداة أو Adapter أو Agent متخصص مع الحفاظ على الواجهات القائمة.
- **تحسين الأمان والمراقبة:** وجود نقاط مركزية للفحص والتسجيل وإدارة دورة التنفيذ.

هذه مزايا تصميمية متوقعة؛ تحقق فائدتها عمليًا يتوقف على جودة التنفيذ والاختبارات والضوابط.

9. أسئلة المناقشة وإجابات عربية نموذجية

س: ما فكرة مشروعك؟

ج: هو نظام ذكاء أعمال قائم على وكلاء ذكيين، يسمح للمستخدم بطرح أسئلة باللغة الطبيعية على قواعد البيانات والوثائق، ثم يخطط للخطوات المناسبة، يسترجع المعلومات ويحللها ويقدم تقريرًا مفهومًا.

س: لماذا استخدمت معمارية وحدات وطبقات؟

ج: لفصل المسؤوليات وتقليل الترابط. لكل وحدة مجال واضح، وتفصل الطبقات بين قواعد المجال وتنسيق الاستخدام والتنفيذ التقني وواجهات API؛ وهذا يسهل الصيانة والاختبار واستبدال المكونات.

س: لماذا لا يتصل الوكيل بقاعدة البيانات مباشرة؟

ج: لأن ذلك يربط منطق الوكيل بتفاصيل قاعدة محددة ويخلط اتخاذ القرار بالاتصال والتنفيذ. أداة DatabaseTool ومحرك Query Engine يقدمان طبقة تجريد تسمح بتغيير Adapter أو المصدر دون تعديل منطق الوكيل كله، كما يوفران نقاطًا أوضح للتحقق والتسجيل.

س: ما وظيفة Query Engine؟

ج: تنظيم دورة الاستعلام: بناء السياق، تصنيف الطلب، اختيار المصدر أو المحول، تنفيذ الاستعلام، ثم تحويل النتيجة إلى صيغة موحدة.

س: لماذا نحتاج Schema Engine؟

ج: لأن النموذج يحتاج معلومات صحيحة عن الجداول والأعمدة والأنواع والعلاقات قبل توليد SQL. توفر وحدة المخطط هذه المعلومات بصيغة منظمة وتقلل التخمين.

س: ما الفرق بين Planner وExecutor؟

ج: Planner يحدد ما المهام المطلوبة وبأي ترتيب، أما Executor فينفذ تلك المهام ويمرر النتائج والسياق بينها.

س: ما وظيفة DatabaseTool؟

ج: هي جسر بين Multi-Agent وQuery Engine، تخفي تفاصيل الاستعلام عن الوكيل وتتيح له طلب بيانات عبر واجهة منظمة.

س: ما وظيفة RAGTool؟

ج: تمنح الوكيل واجهة للبحث في الوثائق واسترجاع محتوى مرتبط بالسؤال من دون أن يعرف الوكيل تفاصيل تجهيز المستندات أو التضمينات أو البحث المتجهي.

س: هل Memory هي قاعدة بيانات المشروع؟

ج: لا. الذاكرة تحفظ سياقًا أو معلومات محادثة يحتاجها الوكيل. قاعدة الأعمال تحفظ السجلات الفعلية، ومخزن المتجهات قد يحفظ فهارس أو تمثيلات للبحث في الوثائق.

س: لماذا وحدة LLM مستقلة؟

ج: لتوحيد طريقة استدعاء النماذج وفصل التطبيق عن مزود بعينه؛ فيمكن اختيار مزود متوافق أو محلي بحسب الإعدادات، مع إبقاء منطق التطبيق موحدًا.

س: كيف تحمون قاعدة البيانات من SQL غير آمن؟

ج: يوجد وفق وصف المشروع SQL Validator قبل التنفيذ. ويجب أن تتكامل هذه الخطوة مع صلاحيات محدودة، وقواعد تسمح بالاستعلامات المطلوبة فقط، وحدود تنفيذ، وعدم الاعتماد على النموذج وحده كضابط أمان.

س: أين قاعدة البيانات النهائية؟ وما دور MongoDB؟

ج: وفق القرار الموضح في الملف، الإنتاج يستخدم Supabase/PostgreSQL. أما MongoDB فكان للتجربة أو Adapter مؤقت لمحرك الاستعلام، وليس قاعدة المشروع النهائية.

س: ماذا يفعل bootstrap.py؟

ج: ينشئ الخدمات ويربط الاعتماديات بينها عند بدء التطبيق؛ أي يعمل كنقطة تركيب للمكونات، بدل أن ينشئ كل جزء تبعياته بطريقة مستقلة ومتشابكة.

10. إجابة أكاديمية موجزة بالإنجليزية

Why this architecture?
“We use a modular, layered architecture based on separation of concerns. The Multi-Agent module handles planning and orchestration; the Query Engine classifies, routes, and coordinates query execution; the Schema Engine provides structured database metadata; the Chat Database module handles the natural-language-to-SQL workflow; the LLM module abstracts model providers; and the Memory module manages relevant conversational context. Database and document-retrieval tools keep the agents decoupled from implementation details. The intended production database is Supabase/PostgreSQL, while MongoDB was used for testing or as a temporary adapter, according to the project description.”

يمكن حفظ هذه الإجابة ثم شرح كل نقطة بمثال، بدل ترديدها فقط.

11. قاموس مختصر للمصطلحات

المصطلح	شرحه بالعربية
API / Router	واجهة يتبادل عبرها التطبيق الطلبات والاستجابات؛ وRouter يجمع مسارات ذات صلة.
Adapter	محول يربط واجهة عامة بمصدر أو تنفيذ محدد.
DTO	كائن منظم لنقل البيانات بين أجزاء النظام.
Entity	كائن يمثل مفهومًا أساسيًا في مجال التطبيق.
Interface / Contract	عقد يحدد العمليات التي يجب على التنفيذ توفيرها.
Factory	مكوّن ينشئ التنفيذ المناسب وفق الإعدادات أو النوع المختار.
Provider	خدمة أو مزود نموذج لغوي، مثل مزود سحابي أو نموذج محلي.
Prompt	تعليمات وسياق يقدمان إلى النموذج لأداء مهمة محددة.
SQL	لغة الاستعلامات المستخدمة للتعامل مع قواعد بيانات علائقية.
Schema	وصف بنية قاعدة البيانات: الجداول والأعمدة والأنواع والعلاقات.
Metadata	بيانات تصف بيانات أخرى، مثل نوع العمود أو اسم الجدول.
Cache	نسخة مؤقتة لتسريع الوصول أو تقليل العمليات المتكررة.

Streaming	إرسال أجزاء الاستجابة تباعًا بدل انتظار اكتمالها كلها.
Logging / Auditing	تسجيل أحداث التشغيل للتشخيص أو المراجعة، مع حماية البيانات الحساسة.
Vector DB / Embedding	قاعدة أو فهرس للبحث الدلالي؛ وEmbedding تمثيل عددي للنص يساعد على مقارنة المعنى.

12. الخلاصة التي ينبغي فهمها

```
المستخدم يطرح هدفًا
  ↓
Agent يفهم الهدف ويخطط
  ↓
Tools: قاعدة بيانات و/أو بحث وثائق
  ↓
Query Engine ينظم الوصول إلى البيانات
  ↓
Schema Engine يوضح بنية الجداول عند الحاجة
  ↓
Chat Database يولد SQL ويتحقق منه وينفذه
  ↓
Supabase / PostgreSQL (بيانات الأعمال المقصودة)
  ↓
Analyzer و يستخرج المعنىReporter يصوغ النتيجة
  ↓
تقرير للمستخدم

Memory: تحفظ سياقًا مناسبًا للمحادثة
LLM Module: يوفر واجهة موحدة للنماذج
bootstrap.py: يركب الخدمات ويربطها
main.py: يشغل تطبيق FastAPI ويعرض API
```

الخلاصة بكلمات بسيطة: المستخدم يطلب، الوكيل يقرر، الأدوات تنفذ، محرك الاستعلام ينظم الوصول، قاعدة البيانات أو الوثائق توفر المحتوى، والنموذج يساعد على فهم اللغة وصياغة SQL أو التقرير. ثم يُعاد الناتج للمستخدم مع الاستفادة من سياق المحادثة عند الحاجة.

> **للاستعداد للمناقشة:** ابدأ بالفكرة العامة، ثم اشرح مسار سؤال واحد، وبعدها وضّح سبب فصل كل مكوّن. لا تحفظ أسماء الملفات وحدها؛ اربط كل اسم بمسؤوليته وبالمكوّن الذي يستدعيه والنتيجة التي يعيدها.

أُعدّ هذا الشرح اعتمادًا على النص المرفق، مع توضيح المصطلحات وتوسيع الفكرة. ينبغي مطابقة أسماء الملفات والتفاصيل التنفيذية مع النسخة الفعلية من المشروع قبل اعتمادها كتوثيق نهائي.